JMETER · GATLING · CAPACITY

Deep dive in practical load and stress testing

How many users the API can hold, and where it gives way

JMeter 5.6.3 · Gatling 3.16 Java DSL
Java 21 · Spring Boot on a test stack

Examples browse a catalogue and sign in.
The same patterns apply to any HTTP API on this stack.

1. What these tests prove

A functional test answers a question about one user. The fee for five items is 450. A blank email does not call the API. Login returns a token. JUnit, Jest, and Cypress already own those questions. A load test answers a different question: at the traffic you expect, does the API stay inside a budget you wrote down before the run?

Write the budget first. For this guide the load budget is: 5 requests a second, 95th-percentile response time under 500 milliseconds, error rate under 1 percent. The numbers are a teaching budget, not a law of the server. Change them when you know the traffic. A run without a budget is a demo.

Run	Shape of the traffic	The question
Smoke	One user, one minute	Does this plan hit the right host and return 200?
Load	The peak you expect, held steady	Does the budget hold for that peak?
Stress	Past the peak, until the curve bends	Where do errors appear, and which resource saturates?
Spike	A sudden step, then back	Does the API recover after a short stampede?
Soak	A moderate rate, for hours	Do pools, disk, or memory give way only after time passes?

A stress run is supposed to hurt. Gating it on the load budget hides the knee, because the build goes red at the first interesting point. Gate the load run. Read the stress run. A soak is the run that finds a connection leak a two-minute test cannot see.

2. The numbers that matter

Throughput is completed requests per second. Latency is how long one request took. Report the median, the 95th percentile, and the 99th percentile. The average hides the slow tail: ninety fast calls and ten calls that took ten seconds can still average under a second.

The error rate counts HTTP 5xx, timeouts, and failed checks. A 200 with an empty body when the check required a product sku is a failure. A 401 storm is a failure of the plan or of the accounts, and it is not a capacity result until you know which.

Saturation is the resource that ran out: CPU, the database connection pool, Redis, disk, or the injector itself. Concurrency is roughly throughput times response time. When the 95th percentile doubles and throughput stays flat, users are waiting in a queue.

```
Budget for the load run
  throughput     about 5 requests / second
  p50            reported, not gated
  p95            under 500 ms
  p99            reported
  errors         under 1 percent

The stress run records the same columns
and marks the rate where errors leave zero.
```

3. Where the virtual users sit

JMeter and Gatling are injectors. They open many HTTP connections to an API that is already running. They do not start Spring, Postgres, or Redis. The functional suite does that, inside Testcontainers, for one user. Point the injector at a test stack with the same shape as production: the same Java version, the same database, the same pool sizes. Aim it at a stack you are allowed to saturate. A run against the live shop is how the shop goes down. A result from an API sharing a laptop with the injector is a rehearsal. The CPU you spent generating traffic was stolen from the server.

Read the diagram from the bug outward. A wrong fee stops in the unit test. A missing login token stops in the functional API test. A 95th percentile over 500 ms at 5 requests a second stops here. So does a pool that only empties after an hour.

4. The scenario, before the tool

Start with the request that does real work. `GET /api/v1/products` reads PostgreSQL and returns a page. The list lives under `content`. A useful check is status 200 and `$.content[0].sku`. That check is honest only when the database has at least one published product. An empty page makes every sample fail, and the error rate measures the seed.

`GET /api/v1/health` is the smoke, one user, to prove the plan talks to the host. It is a poor load scenario. It barely touches the database, so a green health run can sit beside a catalogue that has already fallen over.

The second scenario signs in and then calls a route that needs the token. `POST /api/v1/auth/login` with `email` and `password` returns `accessToken`. The next request sends `Authorization: Bearer` plus that token, for example `GET /api/v1/auth/me`.

The catalogue read is public, so the browse plan sends no token. The profile call is not public. Each virtual user that signs in needs an account whose password is right. A password on this API contains a letter and a digit, and it is at least eight characters. `Secret1a` meets that rule. A correct password clears the failure count. Three wrong passwords lock the account for thirty minutes, and during the lock the correct password still returns 401 with the same detail as a bad password. One shared user and a wrong password turns the run into a lockout test. Put many accounts in a CSV. A lockout scenario is a separate plan, with one account, and it stops at three failures.

```
email,password
ada1@shop.test,Secret1a
ada2@shop.test,Secret1a
ada3@shop.test,Secret1a
```

Between requests, pause. That pause is think time: the person reading the page. A Uniform Random Timer in JMeter, or `pause` in Gatling, spreads the calls. A tight loop with no pause is a different scenario, and it is harsher than the shop. Say which one you ran.

5. Installing each tool

Both tools need the Java 21 JDK this course already uses. Check that before you install either one. JMeter is a program you unpack. Gatling is a Maven project that sits beside the API. Neither one is a dependency of the API, and neither one starts Spring, Postgres, or Redis.

```
java -version
# openjdk version "21..."

export JAVA_HOME="$(/usr/libexec/java_home -v 21)"
echo "$JAVA_HOME"
```

On macOS, JMeter calls `/usr/libexec/java_home` when `JAVA_HOME` is empty, and that can pick an older JDK. The export above pins version 21. On Linux, set `JAVA_HOME` to the JDK 21 directory. JMeter stops when it cannot find a Java home and `java` is not on `PATH`.

5.1 JMeter

Download the binary archive `apache-jmeter-5.6.3.tgz`, or the zip with the same version, from the binaries list at jmeter.apache.org. The source archive has no `bin/jmeter` script. JMeter 5.6.3 runs on Java 8 or newer. Unpack it in a directory whose path has no spaces. The nightly 6.0 snapshot is not this install.

```
tar -xzf apache-jmeter-5.6.3.tgz
export PATH="$PWD/apache-jmeter-5.6.3/bin:$PATH"
jmeter -v
```

`jmeter -v` must print 5.6.3. Add that `bin` directory to your shell profile, or a new terminal will not find the command. `jmeter` with no arguments opens the GUI, and the GUI is only for writing the plan. On Windows, `bin\jmeter.bat` in the same folder opens that GUI. The default heap is already 1 GB. Raise `HEAP` only when the injector log says it ran out of memory. Do not copy this tree into the API repository.

5.2 Gatling

Gatling 3.16.0 is not a program on your `PATH`. The Java DSL is a test-scoped library, and the Maven plugin 4.21.12 runs it. That plugin does not bring Gatling with it. The dependency `gatling-charts-highcharts` 3.16.0 does. Gatling 3.16 needs Java 17 or newer. Make a `loadtest` directory beside the API and put this `pom.xml` in it. Leave the API pom alone, so `./mvnw verify` stays the functional suite.

```
<project>
  <modelVersion>4.0.0</modelVersion>
  <groupId>rw.ac.rca</groupId>
  <artifactId>loadtest</artifactId>
  <version>1.0.0</version>
  <properties>
    <maven.compiler.release>21</maven.compiler.release>
    <gatling.version>3.16.0</gatling.version>
  </properties>
  <dependencies>
    <dependency>
      <groupId>io.gatling.highcharts</groupId>
      <artifactId>gatling-charts-highcharts</artifactId>
      <version>${gatling.version}</version>
      <scope>test</scope>
    </dependency>
  </dependencies>
```

The same file continues with the plugin. Without this plugin, `mvn gatling:test` is not a Maven goal.

```
  <build>
    <plugins>
      <plugin>
        <groupId>io.gatling</groupId>
        <artifactId>gatling-maven-plugin</artifactId>
        <version>4.21.12</version>
      </plugin>
    </plugins>
  </build>
</project>
```

The simulation class from section 7 goes in `src/test/java`. The account CSV goes in `src/test/resources`. You need Maven 3.9. `mvn -version` should show that, and Java 21.

```
loadtest/
  pom.xml
  src/test/java/CatalogueLoadSimulation.java
  src/test/resources/users.csv

cd loadtest
mvn -q test-compile
```

`test-compile` only checks that the simulation builds. It does not call the API. `mvn gatling:test` does call the API, so start the test stack first. The HTML report appears under `target/gatling` after that run.

6. JMeter

JMeter 5.6.3 is the stable release this guide uses. The nightly 6.0 snapshot is not the build you measure with. Java 21 runs this binary. Author the plan in the GUI. Measure from the command line. A listener such as View Results Tree, left open during the real run, spends CPU painting samples and makes the API look slower than it is.

6.1 The plan

- **Test Plan.** Name it catalogue-load. Add the host as a variable, `HOST` = 127.0.0.1 and `PORT` = 8080, so the same file can point at another machine.
- **HTTP Request Defaults.** Protocol http, server `${HOST}`, port `${PORT}`.
- **Thread Group, readers.** The load shape from the next section.
- **HTTP Request.** GET, path `/api/v1/products`.
- **Response Assertion.** Response code equals 200.
- **JSON Assertion.** `$.content[0].sku` exists.
- **Uniform Random Timer.** 200 ms to 1 s, the think time.

The sign-in plan adds a CSV Data Set Config on the thread group. Variable names are `email,password`. Sharing mode is all threads. Recycle is on only when you have enough real accounts. The login sampler is POST `/api/v1/auth/login` with header `Content-Type: application/json` and this body.

```
{"email":"${email}","password":"${password}"}
```

A JSON Extractor saves `$.accessToken` as `token`. The next sampler sends the header `Authorization` with the value `Bearer ${token}`.

6.2 Four shapes from one thread group

Run	Threads	Ramp-up	Scheduler
Smoke	1	1 second	60 seconds, loops forever
Load	20	30 seconds	120 seconds, loops forever
Stress	200	60 seconds	180 seconds, loops forever
Spike	10, then 150	The second group starts after 60 seconds	Each group runs 90 seconds

Loops forever plus a scheduler duration is the hold. A loop count of 1 ends the run when each thread has asked once, which is too short to see a pool fill. The spike uses two thread groups. The second group's scheduler has a startup delay of 60 seconds. That delay is the step. Soak uses the load thread count and a duration of two hours or more. Watch the database pool and the process memory across the whole window, not only the last minute.

6.3 The command that measures

```
jmeter -n \
  -t catalogue-load.jmx \
  -l target/results.jtl \
  -e -o target/jmeter-report
```

`-n` is non-GUI. `-l` writes every sample. `-e -o` builds the HTML dashboard: throughput, percentiles, and the error table. The heap is already 1 GB. Set `HEAP` to a larger value only when the injector log says it ran out of memory.

A failed sample is a row in that report. The `jmeter -n` process still exits 0 when samples fail. Gatling is the tool whose assertion fails the build. To fail a JMeter pipeline, count error rows in the `.jtl` file after the run. Do not treat a green exit code as a passed budget.

7. Gatling

The `loadtest` project from section 5 already has Gatling 3.16.0 and the Maven plugin. This section is the simulation you put in that project. `./mvnw verify` on the API must not open twenty users against a server. Load is a job you start on purpose.

7.1 A load simulation

Save the class as `src/test/java/CatalogueLoadSimulation.java`. `injectOpen` starts users and lets them finish. `rampUsers` spreads the arrival. `constantUsersPerSec` holds the rate. `percentile3` is the 95th percentile in Gatling's assertion API. The assertions are the budget. A breach fails `mvn gatling:test`.

```
import static io.gatling.javaapi.core.CoreDsl.*;
import static io.gatling.javaapi.http.HttpDsl.*;

import io.gatling.javaapi.core.ScenarioBuilder;
import io.gatling.javaapi.core.Simulation;
import io.gatling.javaapi.http.HttpProtocolBuilder;
import java.time.Duration;

public class CatalogueLoadSimulation
        extends Simulation {

  HttpProtocolBuilder httpProtocol = http
      .baseUrl("http://127.0.0.1:8080")
      .acceptHeader("application/json");

  ScenarioBuilder browse = scenario("browse")
      .exec(http("list products")
          .get("/api/v1/products")
          .check(status().is(200))
          .check(jsonPath("$.content[0].sku")
              .exists()))
      .pause(Duration.ofMillis(200),
             Duration.ofSeconds(1));

  {
    setUp(browse.injectOpen(
        rampUsers(20).during(
            Duration.ofSeconds(30)),
        constantUsersPerSec(5)
            .during(Duration.ofSeconds(60))
    )).protocols(httpProtocol)
     .assertions(
        global().successfulRequests()
            .percent().gt(99.0),
        // default config: percentile3 is p95
        global().responseTime()
            .percentile3().lt(500)
     );
  }
}
```

7.2 Stress, spike, and sign-in

A JMeter thread group is closed. Twenty threads means at most twenty requests in flight. If the API slows, throughput falls, because those threads are stuck waiting. `injectOpen` is open. New users arrive on the clock even while earlier ones are still waiting. That is how a public catalogue behaves. The closed injection below is the Gatling shape that matches the thread group.

```
browse.injectClosed(
    constantConcurrentUsers(20).during(
        Duration.ofSeconds(60)));
```

Find the knee with stairs. Each level holds a higher arrival rate long enough to read. Leave the 99 percent assertion off this class. The report is the result: the level at which the error curve leaves zero. A soak replaces the stairs with `constantUsersPerSec(2).during(Duration.ofHours(2))`.

```
browse.injectOpen(
    incrementUsersPerSec(5)
        .times(6)
        .eachLevelLasting(
            Duration.ofSeconds(30))
        .separatedByRampsLasting(
            Duration.ofSeconds(10))
        .startingFrom(5)
);
```

`stressPeakUsers(200).during(Duration.ofSeconds(60))` is the spike, not a hold at 200 concurrent users. It starts 200 users in total. The arrival rate is quiet at the edges and bunched in the middle of the minute. Read whether the 95th percentile comes back down after that burst.

Sign-in feeds the CSV from the classpath, `src/test/resources/users.csv`. `circular()` reuses rows. That is safe when every password is right and you have enough accounts that one lock does not represent the whole shop. `#{email}` is Gatling's expression for a feeder column. The saved token is a session attribute, not a Java variable.

```
// CoreDsl and HttpDsl are already imported
ScenarioBuilder signIn = scenario("sign in")
    .feed(csv("users.csv").circular())
    .exec(http("login")
        .post("/api/v1/auth/login")
        .header("Content-Type",
            "application/json")
        .body(StringBody(
            "{\"email\":\"#{email}\","
            + "\"password\":\"#{password}\"}"))
        .check(status().is(200))
        .check(jsonPath("$.accessToken")
            .saveAs("token")))
    .pause(Duration.ofMillis(200))
    .exec(http("me")
        .get("/api/v1/auth/me")
        .header("Authorization",
            "Bearer #{token}")
        .check(status().is(200)));
```

Run it with `mvn gatling:test` from the load module. The HTML report lands under `target/gatling`. The first seconds include JIT compilation and pool growth. Treat them as warmup. A budget judged only on those seconds measures the warm-up, not the steady rate.

8. The same scenario, two tools

	JMeter 5.6.3	Gatling 3.16
You author	A GUI plan, saved as .jmx	A Java simulation
Virtual user	A thread in the thread group	A user injected into a scenario
Think time	Uniform Random Timer	pause(min, max)
JSON check	JSON Assertion and JSON Extractor	jsonPath in check
Accounts	CSV Data Set Config	csv("users.csv").circular()
Load shape	Threads, ramp-up, scheduler	rampUsers, constantUsersPerSec
Model	Closed. At most N requests in flight	Open with injectOpen. Arrivals keep coming
Stress shape	A larger thread group, ramped	incrementUsersPerSec stairs
Spike	A second group, started after a delay	stressPeakUsers, a burst in the middle
Pass or fail	Read the HTML report. Exit code stays 0.	assertions fail mvn gatling:test
Report	target/jmeter-report from -e -o	target/gatling

Pick either tool for the course and learn the scenario until the report is boring. A second tool that repeats the same catalogue plan teaches the mapping. It does not double the evidence. One honest load run beats two tools aimed at `/health`.

9. Reading a bad report

What you see	What it usually means
Connection refused	The API is down, or HOST and PORT point at the wrong machine. The injector never reached a servlet.
Every login is 401, including the right password	Three wrong passwords locked the account for thirty minutes. The status stays 401. The detail does not change.
JSON path content[0].sku fails on 200	The page is empty. Seed a published product. The error rate is the seed.
p95 climbs, throughput stays flat	A pool or a lock is queueing. Look at the database pool and CPU on the API machine.
Errors only on the injector	The laptop running JMeter ran out of CPU or sockets. Move the injector off the API host and repeat.
The smoke is green and the catalogue is red	Health does not read the catalogue. Keep the products request in the plan.
Soak errors start after an hour	A leak, a log disk, or a pool that never returns connections. The two-minute load run will stay green.

10. What a load test cannot prove

• **The fee.** Five items at 100 cost 450 in the unit test. One sample inside a load run does not pin that rule, and the average would hide a wrong total.

• **A single user's screen.** Cypress and Playwright own the button and the label. The injector never opens that page unless you deliberately drive the dev server, which this guide does not.

• **Production capacity, from a laptop.** The number belongs to the test stack you named: cores, pool size, and dataset. Write those next to the report.

• **A stress run that stayed under the load budget.** You have not found the knee. Raise the peak, or say the knee is above the peak you tried.

• **A shared account with a bad password.** You measured the lockout. That is a security behaviour, and the functional test already covers the third failure.

11. Running one kind of test

```
# functional suite, one user, containers
./mvnw verify

# JMeter load, API already running
jmeter -n -t catalogue-load.jmx \
  -l target/results.jtl \
  -e -o target/jmeter-report

# Gatling load, from the loadtest directory
cd loadtest
mvn gatling:test
```

Keep the load job off the commit pipeline, or make it manual. A push should not aim 200 users at a shared environment. The job that does run states the budget, the dataset, and the machine. Archive the HTML report with the build.

```
load:
  when: manual
  script:
    - cd loadtest && mvn gatling:test
  artifacts:
    paths:
      - loadtest/target/gatling
```

12. Before you call a run finished

- `jmeter -v` prints 5.6.3, and `JAVA_HOME` points at JDK 21.
- `mvn -q test-compile` passes inside `loadtest`, and that dependency is not in the API pom.
- The budget is written down: rate, p95, error rate.
- The target is a seeded test stack, and the injector is not starving it.
- Smoke passed on /api/v1/products, not only on /health.
- Think time is declared.
- Login uses a feeder of real accounts with the right password.
- The load run is gated. The stress run is read for the knee.
- Warmup is not the whole verdict.
- The report names the tool, the version, the duration, and the machine.

13. Questions

One option is correct. Where a question states the teaching budget, that budget is 5 requests a second, 95th percentile under 500 ms, and errors under 1 percent.

1. A load test, in this guide, answers which question?

A. Is the fee for five items 450?
B. At the expected peak, does the written budget hold?
C. Does the Pay button exist?
D. Does one login return a token?

2. The teaching budget gates which figure?

A. The average response time
B. The maximum only
C. The 95th percentile, under 500 ms, and errors under 1 percent
D. Throughput of the health endpoint

3. A stress run is supposed to

A. Climb past the peak until the error curve leaves zero
B. Stay under the load budget or the build fails
C. Replace the unit tests
D. Use one thread

4. A soak is the run that

A. Sends one request
B. Replaces Cypress
C. Disables think time
D. Holds a moderate rate for hours, looking for leaks and full disks

5. GET /api/v1/health is a weak load scenario because

A. It returns JSON
B. JMeter cannot call GET
C. It barely touches the database, so the catalogue can be down while health is green
D. It requires a token

6. $.content[0].sku fails on HTTP 200 when

A. The catalogue page is empty. The database was not seeded
B. The injector is on another machine
C. Think time is 200 ms
D. The assertion uses p95

7. One shared user and a wrong password, under 200 threads, measures

A. Peak throughput of login
B. The fee rule
C. The Vite proxy
D. The lockout. Three failures lock the account for thirty minutes

8. The login JSON field this API returns is

A. token
B. accessToken
C. jwt
D. session

9. Think time in the plan represents

A. A sleep inside the Spring controller
B. The database pool timeout
C. The pause a person spends between requests
D. The JIT warmup

10. JMeter's real measurement is

A. The GUI with View Results Tree enabled
B. jmeter -n, writing a jtl and an HTML report
C. A unit test with MockMvc
D. Cypress

11. After a JMeter run where half the samples failed, the process exit code is

A. Always non-zero
B. 2
C. 137
D. Still 0. Count the error rows in the jtl if the pipeline must fail

12. Gatling's percentile3() in an assertion is

A. The average
B. The maximum
C. The 95th percentile, with the default configuration
D. Throughput

13. global().successfulRequests().percent().gt(99.0) belongs on

A. The load simulation
B. The stress simulation, so the knee fails the build
C. The unit test of the fee
D. A Cypress screenshot

14. stressPeakUsers(200).during(60 seconds) starts

A. 200 users held concurrent for the whole minute

B. 200 users in total, most of them in a burst in the middle
C. 200 requests a second for a day
D. 200 Jest tests

15. csv("users.csv").circular() is safe when

A. The file has one wrong password
B. The file is empty
C. Every row is a real account with the right password
D. It is attached to the health request

16. A laptop that runs both JMeter and the API

A. Is a rehearsal. The injector steals CPU from the server
B. Is the production capacity number
C. Removes the need for a seed
D. Makes p95 meaningless only in Gatling

17. mvn gatling:test should live

A. Inside ./mvnw verify, on every commit
B. In the Jest script
C. In the Flyway migration
D. In a loadtest directory beside the API, started on purpose

18. The first seconds of a Gatling report are

A. The capacity number
B. Warmup: JIT and pool growth. Judge the steady section
C. Always excluded by percentile3
D. The soak

19. Which bug belongs in this guide's load run?

A. Five items billed at full price
B. A label with no htmlFor
C. p95 above 500 ms at 5 requests a second
D. A private method's name

20. A green stress run that never left the load budget means

A. The knee is above the peak you tried, or you have not raised it yet
B. The API has no knee
C. Think time was too long to count
D. JMeter's exit code failed

21. JMeter 5.6.3 is installed by

A. Adding it to the API pom
B. Unpacking the binary archive and putting its bin directory on PATH
C. npm install jmeter
D. The Gatling Maven plugin

22. mvn -q test-compile inside the loadtest directory

A. Sends the virtual users at the API
B. Replaces ./mvnw verify
C. Compiles the simulation and does not call the API

D. Installs the JMeter GUI

14. Answers

1. B. The fee, the button, and the token are functional tests. Load checks the budget at the expected peak.

2. C. The average hides a slow tail. p95 and the error rate are the gate. p50 and p99 are reported.

3. A. Gating stress on the load budget stops the run at the first interesting point. Read the knee. Gate the load run.

4. D. A two-minute load run stays green while a pool leak shows up only later.

5. C. Use health as a one-user smoke. Put GET /api/v1/products in the plan you judge.

6. A. The path expects a published product. An empty page makes the error rate measure the seed.

7. D. Feed many accounts, each with the right password. A lockout plan is separate and stops at three failures.

8. B. TokenResponse exposes accessToken. The extractor and the Bearer header use that name.

9. C. A Uniform Random Timer or Gatling pause is that gap. A tight loop is a harsher scenario. Say which you ran.

10. B. Listeners left on during the run spend CPU and slow the picture. Author in the GUI. Measure with -n.

11. D. The report holds the failures. The engine ends a normal non-GUI run with System.exit(0).

12. C. The default gatling.conf sets percentile1 to 50, percentile2 to 75, percentile3 to 95, and percentile4 to 99.

13. A. That line is the budget. The stress class records the knee and does not use this gate.

14. B. The arrival rate is quiet at the edges and bunched in the middle. It is not 200 users in flight for the whole minute.

15. C. Circular reuse repeats those accounts. A wrong password locks them.

16. A. When the number matters, run the injector on another machine and write the hardware next to the report.

17. D. The functional suite is one user. A push should not aim a peak at a shared API.

18. B. Judge the rate after the pools are warm. Say so next to the budget.

19. C. The fee is a unit test. The label is a component or Cypress test. The percentile is the load budget.

20. A. State the peak you reached. Raise the stairs if you still need the knee.

21. B. jmeter -v must print 5.6.3. JAVA_HOME must point at a JDK. The source archive has no bin/jmeter script.

22. C. test-compile checks the build. mvn gatling:test is the command that sends the users, and the API must already be running.